

DATA/CS 172: Python for Data Analysis

Data Files in Python

Prof. Travis Mandel

8/30/2024



UNIVERSITY
of HAWAI'I®

HILO

Reminder

- Monday is a holiday
 - No class
- Today is the deadline for applying to the AI research assistant position
 - Send me an email with transcript, a paragraph about why you're interested, a paragraph about your experience, and what modality you would prefer

Today

- Comment wrap-up
- Basics of loading data into Python
- Basics of writing out processed data with Python

Multi-line comments



- Last time we learned comments start with #
- Advanced: can (kind of) do multi-line comments with three quotes `'''`
 - Demo

Supposed to be a class on **Data Analysis** with Python...

- Have not touched any data yet!
- LOTS more to learn about Python
 - Don't worry, we'll get to this
- But since this is a data science class, we need to get started with data right away!



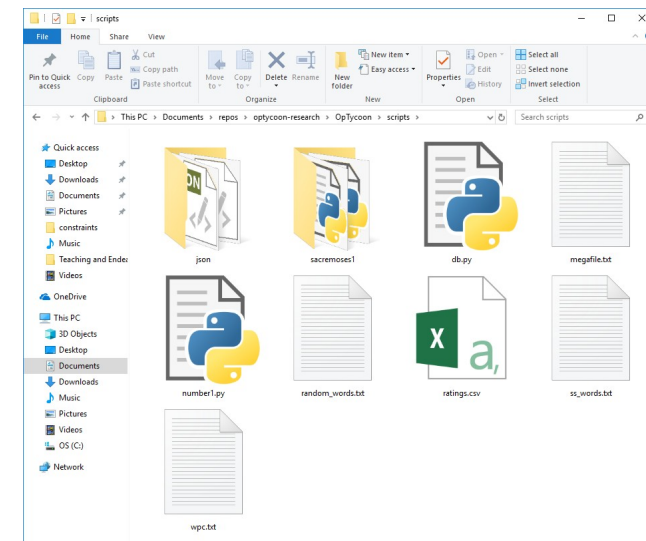
Where does data come from?

- Hard coding
 - Usually, not good!
- User typing it in by hand
 - Example: `print("What's the current rainfall reading?")`
 - Not very common:
 - Doesn't scale to a lot of data
 - If they type interactively, difficult to reproduce
- Data files
- Web APIs
- Databases



Data Files

- Stored somewhere on disk
- Could be in any format - .txt, .xlsx, .csv, .json, etc.
- We'll talk about more advanced formats later- for now just assume text (.txt)
- Two important things
 - **Reading:** Get all the text out of the file ← Important term!
 - **Writing:** Put new stuff in the file ← Important term!
 - Why could writing be important in data science?
 - For example, processing data before handing it off to someone else
 - Original File → Python Script → Processed file



Steps to working with a file

- Step 1: Open the file
- Step 2: Read (or write) the contents of the file
- Step 3: Close the file

Step 1: Open the file

```
myfile = open("example.txt", <mode>)
```

What is <mode>?

“r” - read

“w” – write, and if not there, create

w overwrites anything there already

“x” – similar to “w”, but give error if file already exists

“a” – append output to end of file (useful for logs)

Remember these!



Step 2: Reading/writing



- Read the file
 - `contents = myfile.read()`
 - Reads **the whole file** into the contents variable
 - Probably not what we want with big files... we will soon learn better ways to read files!
- Or...Write to the file
 - Seems similar to printing
 - `print("Hello file", file=myfile)`
 - The `file=` should always be the last thing in the print
 - There's an old way called "write" but it is not usually a good idea
- Remember: The file has to be in the right mode for what you want to do!
 - Doesn't usually make sense to read and write at the same time

Step 3: close the file

- `myFile.close()`
- Very important: If file left open it can prevent others from accessing it!
 - So (in general) **files should be closed as soon as possible**

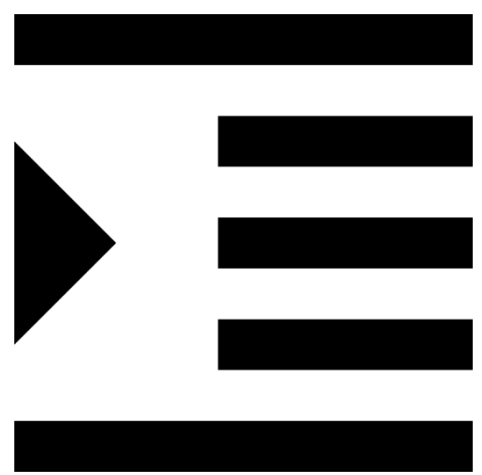


PLEASE CLOSE
THE DOOR

Demo

Another way

- Forgetting to close is a nasty problem
- Some people prefer to use “with” style so they don’t forget with open(...) as f:
 contents = f.read()
- No need for close because can’t use f outside the indented lines!
- The indent is important!
- Doesn’t work in every situation, but can be pretty handy



Important note

- What if we want to read a file multiple times?
 - Why would you want to do this? Sometimes it's more efficient way to write code
 - But this is a bit tricky....
- Idea1:
- `f = open()`
- `contents = f.read()`
- `contents2 = f.read()`
- Doesn't work! See demo for solution

Lab 2 out!